

1.1.2 La démarche DevOps

EN BREF

La **DevOps** sur Hublot, c'est faire circuler le code du poste développeur jusqu'à la **production Netlify** (et optionnellement le **NAS Synology**) de façon **automatisée, contrôlée et documentée**. Le développeur pousse sur Git ; la **CI** vérifie la qualité ; la **CD** déploie seulement si les checks passent. Ce chapitre décrit cette chaîne réelle, pourquoi elle est structurée ainsi, et comment la reproduire.

Objectif pédagogique

Comprendre la **démarche DevOps** : rapprochement développement et exploitation, automatisation, intégration continue, livraison continue, et retour d'information rapide.

Contexte — projet Hublot

Hublot est une **application web statique** (React + Vite) hébergée sur **Netlify**, avec des données RH issues d'exports Excel convertis en JSON. Les mises à jour sont fréquentes (données, filtres, sécurité). Sans pipeline DevOps, chaque changement imposerait des tests manuels répétitifs et des déploiements risqués. Nous avons donc séparé clairement :

- **CI (Intégration continue)** — Est-ce que ce code est sain ? → GitHub Actions
- **CD (Livraison continue)** — Publions-nous cette version ? → Netlify (webhook + gate CI)

Définitions

Terme	Définition
DevOps	Culture et pratiques unissant développement et exploitation : automatiser build, tests et déploiement.
CI (Continuous Integration)	Intégration fréquente du code dans une branche partagée, avec vérifications automatiques à chaque changement.
CD (Continuous Delivery / Deployment)	Livraison ou déploiement automatique d'une version validée vers un environnement cible.
Pipeline	Enchaînement d'étapes (lint → tests → build → déploiement).
Gate (required check)	Blocage du déploiement tant qu'un workflow CI n'est pas vert.
IaC (Infrastructure as Code)	Configuration d'infra versionnée (<code>netlify.toml</code> , Docker, workflows YAML).

Ce qui a été mis en place

Chaîne principale

CODE

```
Développeur (commit/push)
→ GitHub (dépôt Meriem1403/Hublot)
→ Workflow CI (.github/workflows/ci.yml)
  lint → 48 tests → audit npm → build → E2E Playwright
→ Si CI verte + branche main
→ Netlify build & deploy (netlify.toml, dossier build/)
→ Production : https://dirmhublot.netlify.app
```

Workflows GitHub Actions

Fichier	Nom dans GitHub	Rôle
<code>ci.yml</code>	CI	Qualité bloquante à chaque push/PR
<code>cd-netlify.yml</code>	CD Netlify	Hook optionnel post-validation
<code>cd-nas.yml</code>	CD NAS Synology	Déploiement SSH/rsync (semi-automatique)
<code>audit-scheduled.yml</code>	Audit npm planifié	Scan hebdomadaire des dépendances
<code>codeql.yml</code>	CodeQL	Analyse statique sécurité (SAST)
<code>security-scan.yml</code>	Security scan	Gitleaks + Trivy

Pratiques DevOps actives

Pratique	État	Détail
Versionnement Git	En place	Historique, branches <code>main</code> / <code>staging</code>
CI qualité	En place	48 tests Vitest, ESLint, audit prod, build, E2E
CD cloud	En place	Netlify, déploiement après checks requis
Staging	En place	Branche <code>staging</code> + previews (voir 1.2.3)
CD NAS	Semi-auto	<code>scripts/deploy-nas.sh</code> + secrets SSH
Monitoring	En place	<code>/health.json</code> , Sentry optionnel (voir 1.1.2.1)
laC	Périmètre défini	<code>netlify.toml</code> , Docker, workflows
Terraform / Kubernetes	Hors scope	Front statique, pas de cluster

Pourquoi ces choix

Décision	Raison
CI et CD séparés	GitHub Actions ne publie pas directement : Netlify reste maître du déploiement, avec une gate sur la CI. On évite de déployer du code non testé.
Audit npm bloquant	Une dépendance vulnérable en prod exposerait des données RH ; <code>npm run audit:prod</code> stoppe la CI si critical/high non maîtrisé.
E2E sur le parcours login	L'authentification est le premier garde-fou ; un test Playwright garantit qu'un utilisateur non connecté n'accède pas au tableau de bord.
Double cible Netlify + NAS	Netlify = accès cloud pour la démo et la prod Studi ; NAS = hébergement institutionnel possible en réseau interne.
Pas de Terraform	Application statique : <code>netlify.toml</code> + Docker suffisent pour le périmètre actuel.

Comment ça fonctionne

Étape 1 — Développement local

Le développeur travaille sur une branche, lance l'app en DEV (`npm run dev`) et exécute les mêmes commandes que la CI avant de pousser :

TERMINAL

```
npm run lint # ESLint – qualité de code
npm run test:run # 48 tests Vitest
npm run audit:prod # Vulnérabilités npm (prod)
npm run build # Artefact deployable (dossier build/)
npm run test:e2e # Playwright – parcours login
```

Étape 2 — Push et CI

Chaque push sur `main` ou `staging` (ou PR) déclenche le workflow **CI**. Si une étape échoue, le développeur reçoit une alerte GitHub et **Netlify ne déploie pas** (option Deploy only when required checks pass).

Étape 3 — Déploiement Netlify

Netlify lit `netlify.toml` : commande `npm run build`, publication du dossier `build/`, redirections SPA, en-têtes de sécurité. Les identifiants (`VITE_APP_USERNAME`, `VITE_APP_PASSWORD`) sont des **variables d'environnement** Netlify, jamais dans Git.

Étape 4 — Déploiement NAS (optionnel)

Le script `scripts/deploy-nas.sh` rebuild, teste, puis rsync vers le Synology. Le workflow `cd-nas.yml` peut être lancé manuellement (`workflow_dispatch`) avec secrets SSH.

Étape 5 — Retour d'information

- Échec CI → correction avant merge
- `/health.json` en prod → sonde de disponibilité (1.1.2.1)
- Sentry (si DSN configuré) → erreurs JavaScript en production

Quand

Événement	Comportement
Push / PR sur <code>main</code> , <code>staging</code>	CI exécutée (≈ 5–10 min)
CI verte sur <code>main</code>	Netlify déploie automatiquement
Push sur branche feature	CI + éventuellement Deploy Preview Netlify
Hebdomadaire	Audit npm planifié, CodeQL, security-scan
Manuel	<code>workflow_dispatch</code> pour CD NAS

Preuves et démonstration

Voir la CI en action : github.com/Meriem1403/Hublot/actions (<https://github.com/Meriem1403/Hublot/actions>) → workflow **CI** → jobs lint, tests, audit, build, e2e.

Vérifier la prod après deploy :

TERMINAL

```
curl -s https://dirmhublot.netlify.app/health.json  
npm run security:headers https://dirmhublot.netlify.app
```

SYNTHÈSE

« Hublot applique une DevOps **documentée et vérifiable** : le code passe par une **CI bloquante** (lint, 48 tests, audit, build, E2E) avant tout déploiement **Netlify**. Développement et exploitation ne sont pas silos : la même chaîne tourne en local et sur GitHub, avec traçabilité dans Actions et Netlify Deploys. »

Documents liés

- › 1.1.1 Les méthodes Agile — user stories et DoD
- › 1.1.2.1 Monitoring et supervision — health.json, Sentry
- › 1.1.3 Les bases d'un environnement de test — DEV, TEST, STAGING, PROD
- › 1.1.4 La mise en place de l'intégration continue (CI).md) — détail du workflow CI
- › 1.1.5 La mise en place de la livraison ou déploiement continu (CD).md) — Netlify et NAS
- › 1.2.4.2 Sécurité du déploiement — HTTPS, variables, headers